

07/07/2026 TP3 : Machine à état.

L'objectif de ce TP est de réaliser une architecture permettant de faire clignoter deux LEDs RGB en rouge, vert et bleu. Le pilotage des LEDs se fera à l'aide de machines à états. Lors de ce TP vous apprendrez à utiliser des paramètres génériques ainsi que des modules

1/ Dans un fichier .vhd, créez un module Counter_unit à partir du compteur du TP1. Le module prendra en entrée un signal d'horloge et de resetn, et donnera en sortie le signal end_counter. Utilisez un paramètre generic() pour définir le nombre de coup d'horloge à compter.

Lorsqu'on souhaite importer une **entity** déclarée dans un autre fichier, il faut déclarer dans le **behavioral** avant le begin un **component** avec les différentes entrées/sorties puis l'**instancier** après le begin.

2. En schéma RTL, créez un compteur du signal end_counter. Ce compteur doit permettre de déterminer le nombre de cycles allumé/éteint qui ont été effectués par la LED. Le compteur doit pouvoir être remis à 0, maintenir sa valeur actuelle ou s'incrémenter.

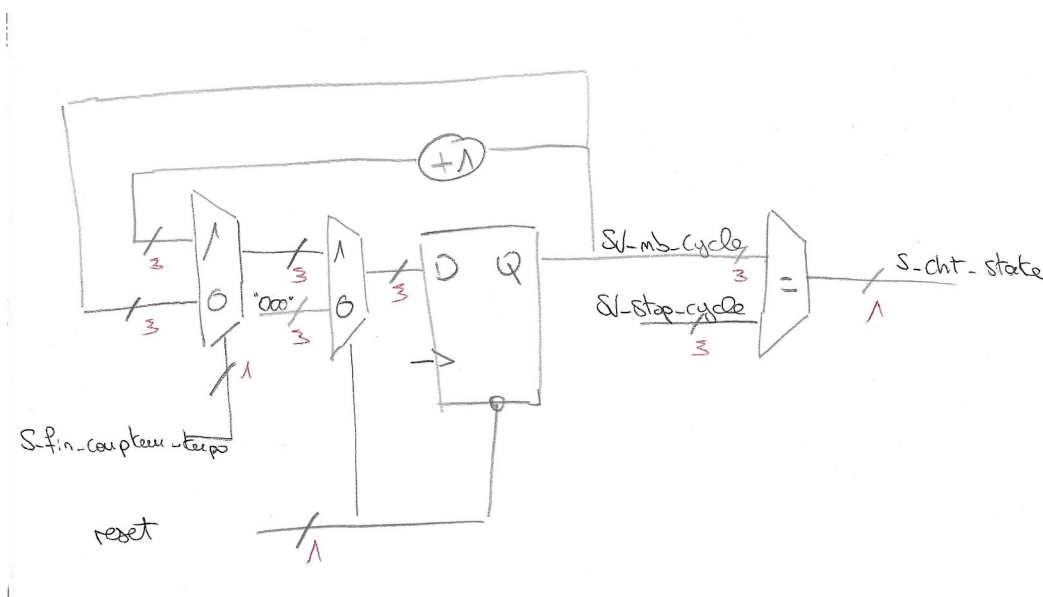


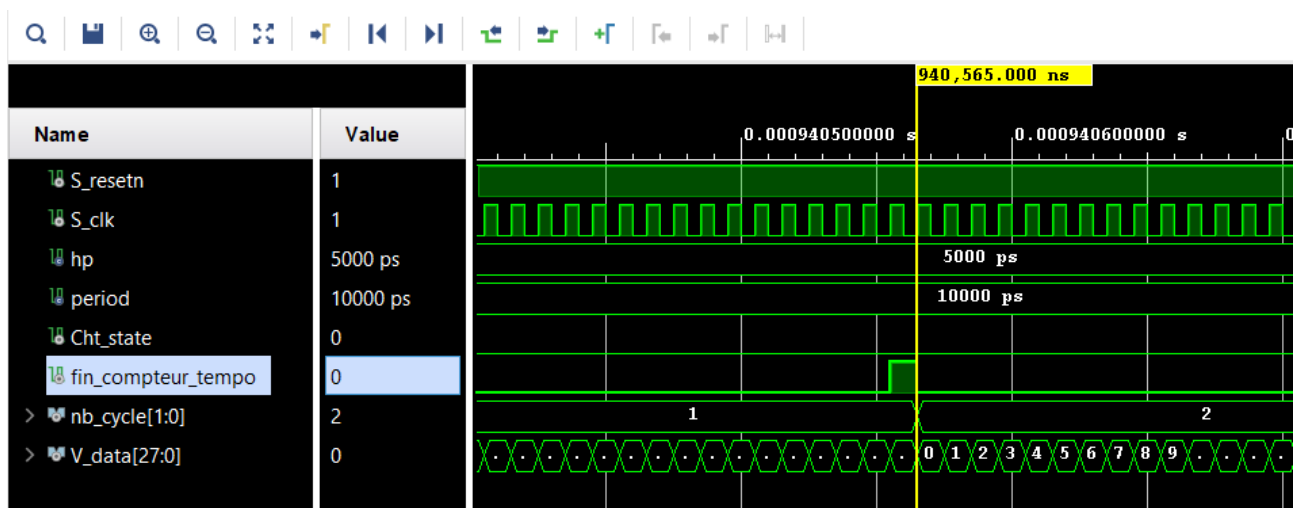
Figure 1 : Schéma RTL du compteur de cycle.

3. Ecrivez un code VHDL décrivant ce compteur de cycle, vous utiliserez le module Counter_unit.

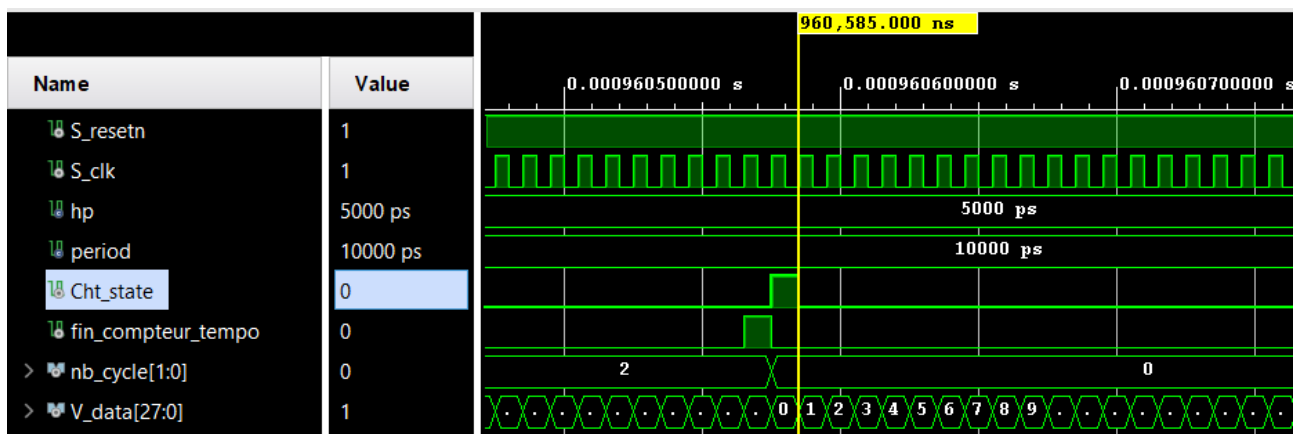
4. Tester votre architecture avec un testbench.

Au départ, le compteur avait été écrit pour se réinitialiser après 3 cycles de temporisation. Ensuite, le système à été modifié pour être réinitialisé après 6 cycles.

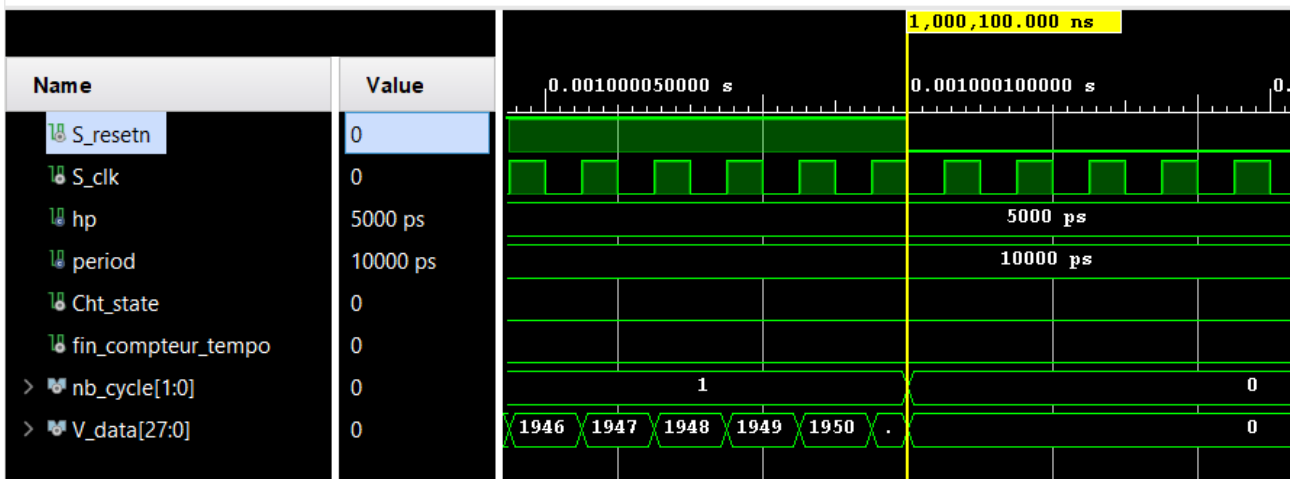
Élément testé	Critère de succès	Ok/Nok
Signal de changement d'état cht_state	- nb_cycle s'incrémente à chaque fin_compteur_temporisation - nb_cycle se remet à 0 lorsque nb_cycle atteint 2. - cht_state = 1 quand nb_cycle atteint 2	OK
Resetn	- Nb_cycle et V_data = 0 quand Resetn vaut 0	OK



nb_cycle s'incrémente à chaque front montant de fin_compteur_temporisation.

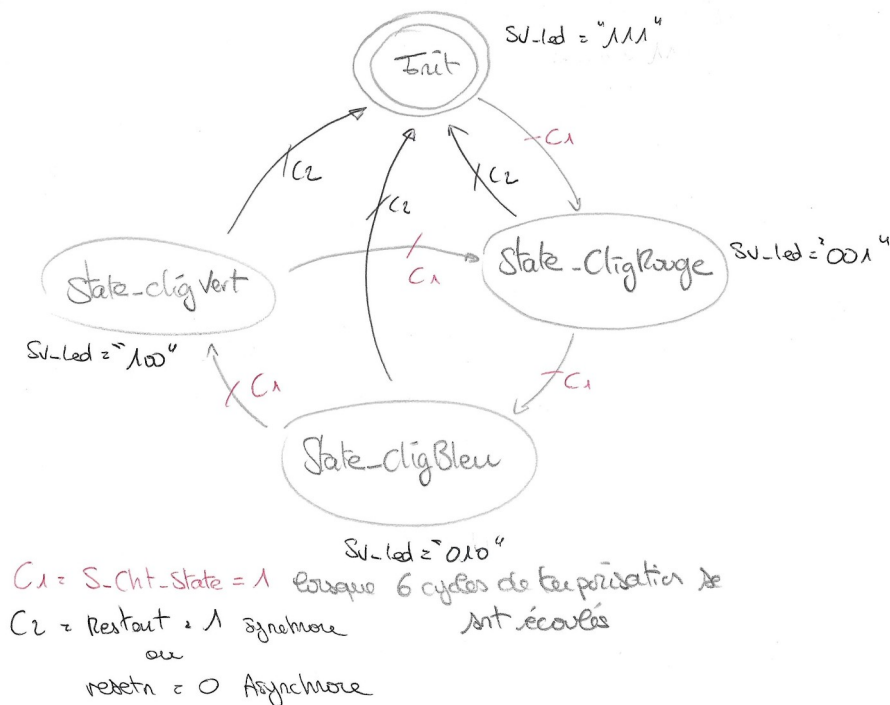


nb_cycle se réinitialise lorsque nb_cycle atteint 2. Cht_state passe à l'état haut pendant un coup d'horloge.



Lorsque S_resetn passe à 0, V_data (le Vecteur interne du compteur de temporisation) passe à 0 ainsi que nb_cycle.

5. Créez en RTL une machine à états (FSM) permettant de faire clignoter une LED RGB en rouge puis bleu et enfin en vert avant de recommencer le cycle (rouge, bleu, vert, ...). Dans chaque état la LED devra clignoter 3 fois. De plus, si le bouton restart est appuyé, on retourne dans l'état initial quel que soit l'état dans lequel on se situe. L'état initial est l'état dans lequel on se situe au démarrage, on passe à l'état rouge après 3 clignotements de la LED en blanc (rouge, vert et bleu actifs en même temps).



Il y a donc quatre états : init, rouge, bleu, vert. C'est la raison pour laquelle le vecteur nb_cycle est sur deux bits.

6. Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

Signaux d'entrée :

- Clk, : Signal d'horloge.
- resetn : Lorsqu'il vaut 1, le système retourne à l'état initial, les vecteurs se réinitialisent.
- restart : lorsqu'il vaut 1, le système retourne à l'état initial.

Signaux de sortie :

- V_Led : Vecteur de commande des leds.

Signaux interne :

- SV_nb_cycle : Vecteur s'incrémentant à chaque état haut du signal de fin de temporisation.
- SV_stop_cycle : Constante, Vecteur contenant la valeur de nb_cycle permettant de

changer d'état.

- S_cht_state : Signal indiquant un changement d'état.
- S_fin_compteur_tempo : Signal indiquant la fin du compteur de temporisation

- SV_led : Vecteur 3 bits décrivant chaque état des leds RGB : Si SV_led = « 001 », c'est la led rouge qui sera allumé, s'il faut « 010 », c'est la bleu, s'il vaut « 001 » c'est la verte, enfin si le vecteur vaut « 111 », la led sera blanche.

- SV_onoffLed : Vecteur 3 bits permettant de contrôler l'allumage ou l'extinction des leds.
- current_state : état dans lequel se trouve actuellement le système.
- next_state : état dans lequel on passera au prochain coup d'horloge.
- S_resetn : Signal permettant d'utiliser le bouton qui est normalement ouvert. (utilisation d'un not pour inverser la logique)

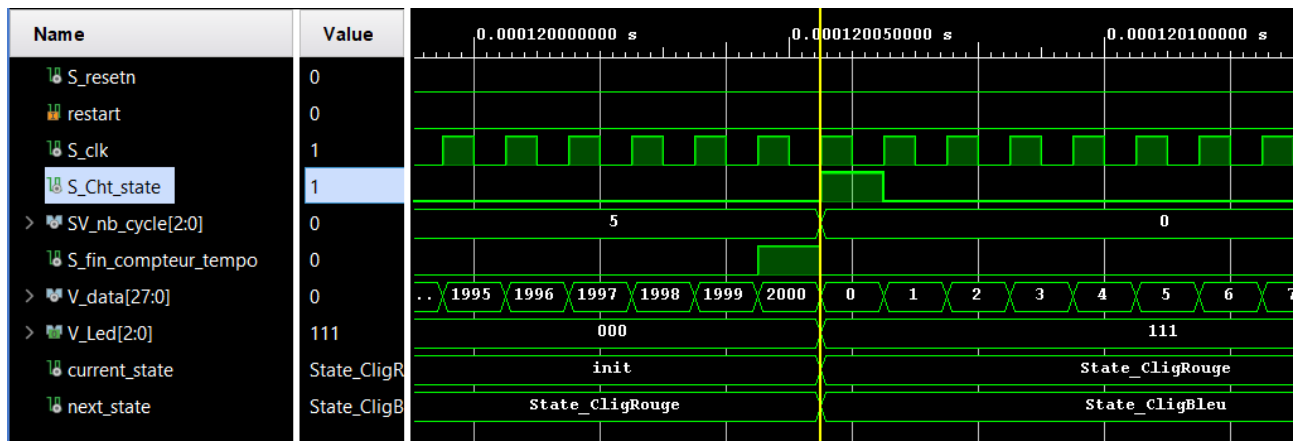
7. Ajoutez à votre code VHDL les éléments que vous venez de créer.

Remarque : Dans une première version, j'allumais les diodes que pendant un coup d'horloge ce qui ne permettait pas de les voir (temps d'allumage 20ns), il faut désormais 6 cycles de temporisation pour passer à l'état suivant car les leds sont éteintes et allumées à chaque fin de temporisation. Ajout d'un signal permettant d'allumer et d'éteindre les leds à l'aide d'un ET logique avec le Vecteur permettant de choisir quelles leds allumer.

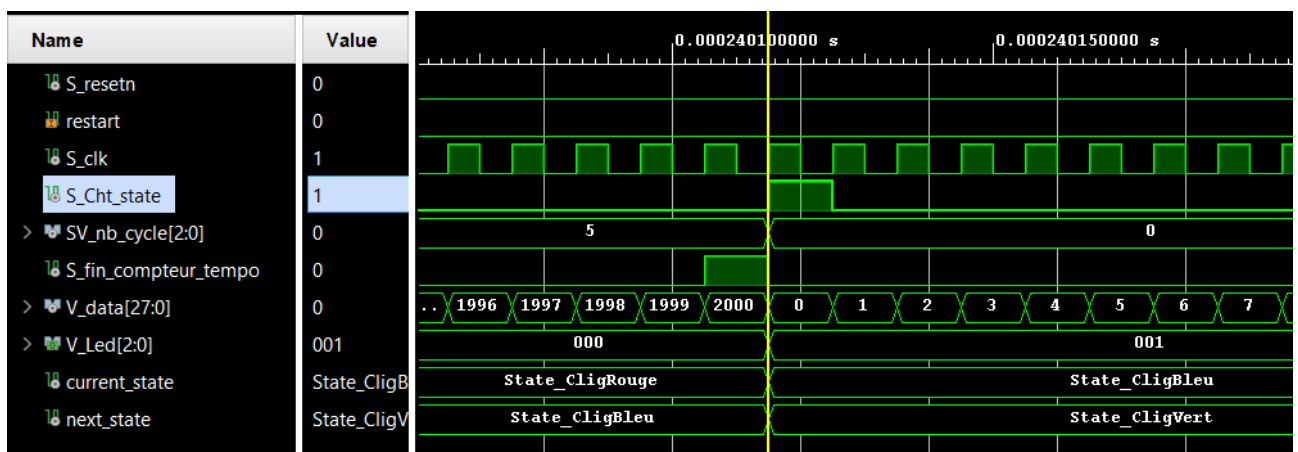
8. Ecrivez un testbench pour tester votre architecture. Vérifiez à la simulation que vous obtenez le résultat attendu.

Élément testé	Critère de succès	Ok/Nok
Ordre des états	Init => State_CligRouge => State_CligBleu => State_CligVert => State_CligRouge.	OK
Choix des leds allumé	SV_led = « 111 » dans init, SV_led = « 001 » dans State_CligRouge SV_led = « 010 » dans State_CligBleu SV_led = « 100 » dans State_CligVert	OK
Contrôle de l'allumage de leds	SV_onoffLed = not SV_onoffLed à chaque fois que S_fin_compteur_tempo = 1	OK
Changement des états	Passage à l'état suivant lorsque S_cht_state = 1 S_cht_state = 1 après 6 cycles.	OK
Reset Asynchrone	Lorsque Reset vaut 0 : SV_onoffLed <= "000"; extinction des leds. current_state = « init » retour à l'état initial S_cht_state = '1' prise en compte immédiate du changement d'état. V_data = « 0 » réinitialisation du compteur de temporisation SV_nb_cycle = « 0 » réinitialisation du compteur de cycle.	OK
Restart Synchrone	Lorsque Restart vaut 1 : SV_onoffLed <= "000"; extinction des leds. current_state = « init » retour à l'état initial S_cht_state = '1' prise en compte immédiate du changement d'état. V_data = « 0 » réinitialisation du compteur de temporisation SV_nb_cycle = « 0 » réinitialisation du compteur de cycle. nb_cycle = « 0 » réinitialisation du compteur de cycle.	OK

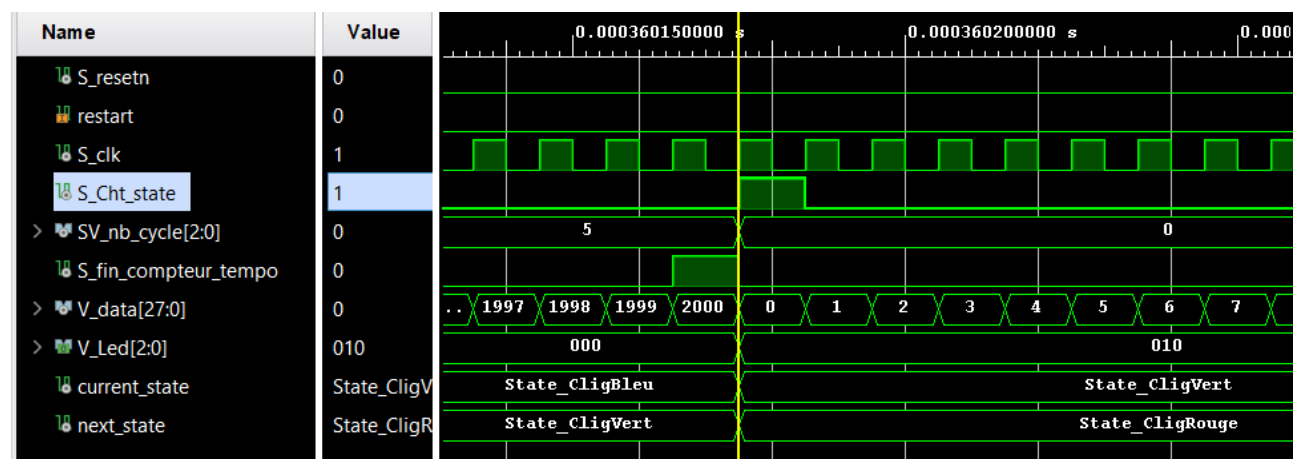
1/ Validation Ordre des états & Changement d'état :



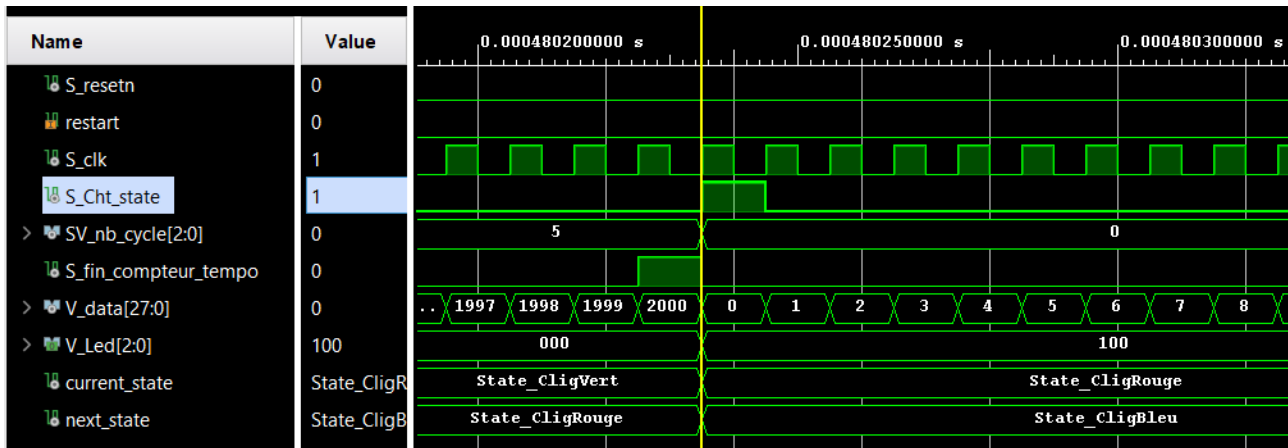
Premier état haut de `S_cht_state`, passage de `init` à `State_cligRouge`.



Second état haut de `S_cht_state`, passage de `State_cligRouge` à `State_clignBleu`.

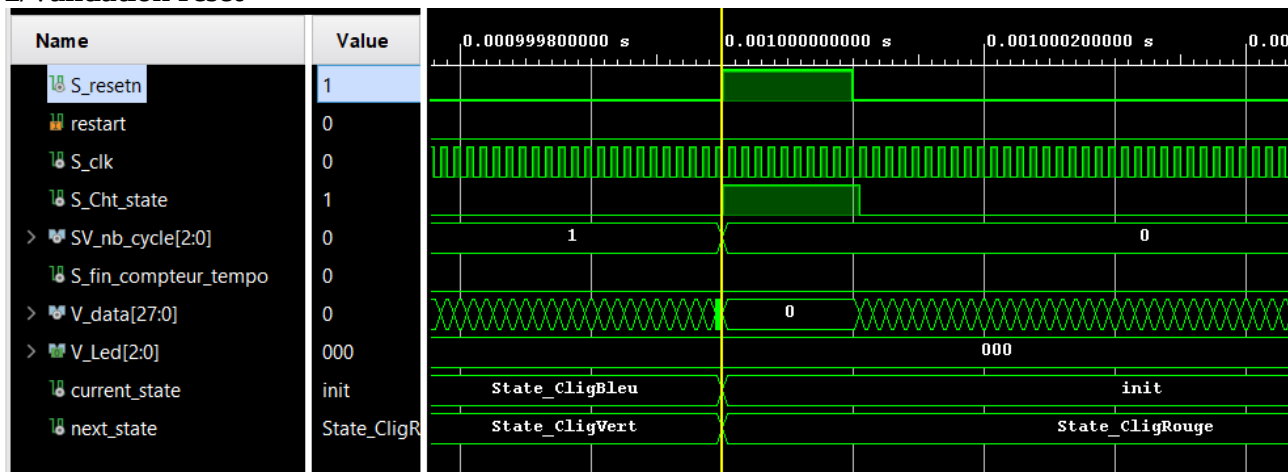


Troisième état haut de `S_cht_state`, passage de `State_clignBleu` à `State_clignVert`.



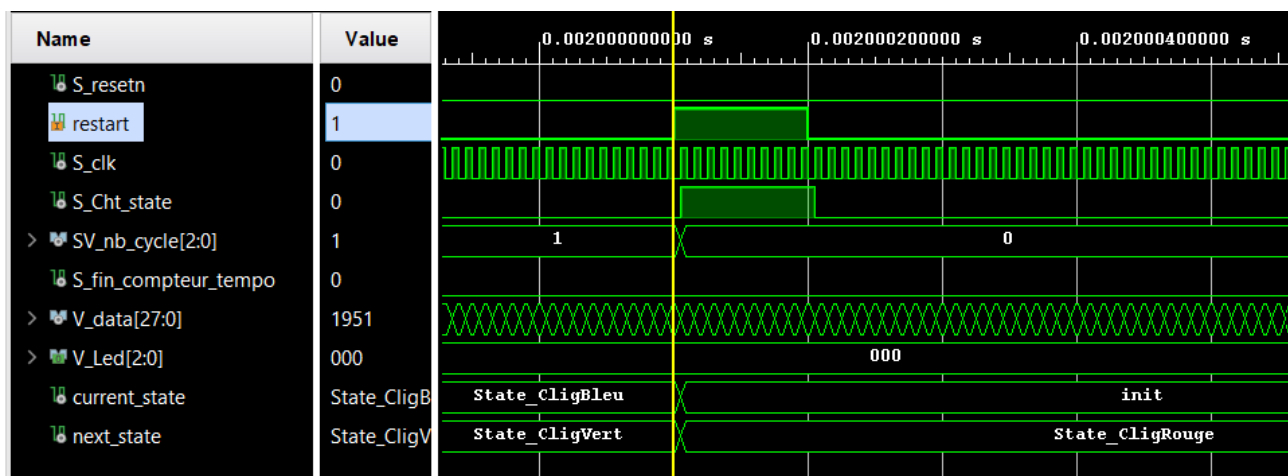
Quatrième état haut de S_cht_state, passage de State_clignVert à State_clignRouge

2/Validation reset



Sur pression du bouton reset (S_resetrn), V_data = 0, current_state = init, next_state = clignRouge, nb_cycle = 0 et S_cht_state = 1. Reset est bien synchrone.

3/Validation restart



Sur pression du bouton restart, V_data = 0, current_state = init, next_state = clignRouge, nb_cycle = 0 et S_cht_state = 1. restart est bien asynchrone.

9. Exécutez la synthèse et relevez les ressources utilisées (y compris la FSM). Sur la schématique, identifiez où se situe votre compteur de cycle.

Cf Schématique en pdf joint.

A/ Sur le schématique, on observe à gauche l'horloge, le reset et le restart suivi de leur buffer et à droite les trois leds en sortie.

B/ À l'intérieur de l'élément counter_unit, on observe les différents registres permettant de stocker la valeur de V_data ainsi que la lut permettant de générer le signal indiquant la fin de la temporisation.

C/ Le compteur de cycle est composé de trois luts et de trois registres de mémorisation surlignés en rose sur la capture suivante. On remarque que les états sont stockés dans deux bascules surligné en rouge.

D/ Les leds sont contrôlées à l'aide d'un AND logique réalisé entre un vecteur 3 bits (un pour chaque leds) et un vecteur 3 bits « 111 » ou « 000 » qu'il est commode d'inverser à l'aide d'un **not**, il fait office d'interrupteur. On peut voir dans le schématique ce AND logique en orange et le vecteur « interrupteur » en vert. Quant au vecteur de sélection des leds, il est issu des bascules surlignées en rouge de la machine à état.

7	Report Cell Usage:		
8	+-----+-----+-----+		
9		Cell	Count
0	+-----+-----+-----+		
1	1	BUFG	1
2	2	CARRY4	7
3	3	LUT2	36
4	4	LUT3	4
5	5	LUT4	3
6	6	LUT5	2
7	7	LUT6	3
8	8	FDCE	36
9	9	FDRE	3
0	10	IBUF	3
1	11	OBUF	3
2	+-----+-----+-----+		
~			

La temporisation utilisée est de 2 secondes, 200 000 000 coup d'horloge stockable sur 28 bits, c'est la raison pour laquelle il y a 7 sommateurs 4 entrées, sur le schématique on peut compter que 28 registres servent à stocker V_data. Il est possible de fixer la taille du V_data manuellement, ce qui ne génère pas d'erreur.

Il y a trois registres pour stocker le nombre de cycles car on compte 6 cycles de temporisation, deux registres pour stocker les états de la FSM et trois registres pour les diodes de sorties,

10. Modifiez le fichier de contraintes pour connecter vos entrées / sorties du système avec les broches de la carte. Réglez l'horloge pour que sa fréquence soit à 100MHz.

Par défaut, l'argument période de l'horloge dans le fichier de contrainte est fixée à 8.0 ns soit une fréquence de 125 MHz, pour avoir une fréquence de 100MHz, on modifie l'argument à 10 ns.

11. Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique).

Max Delay Paths

```
Slack (MET) : 5.425ns (required time - arrival time)
Source: counter_unit_1/V_data_reg[11]/C
(rising edge-triggered cell FDCE clocked by sys_clk_pin {rise@0.000ns fall@4.000ns period=10.000ns})
Destination: counter_unit_1/V_data_reg[25]/D
(rising edge-triggered cell FDCE clocked by sys_clk_pin {rise@0.000ns fall@4.000ns period=10.000ns})
Path Group: sys_clk_pin
```

Setup

Worst Negative Slack (WNS): 5,425 ns
 Total Negative Slack (TNS): 0,000 ns
 Number of Failing Endpoints: 0
 Total Number of Endpoints: 48

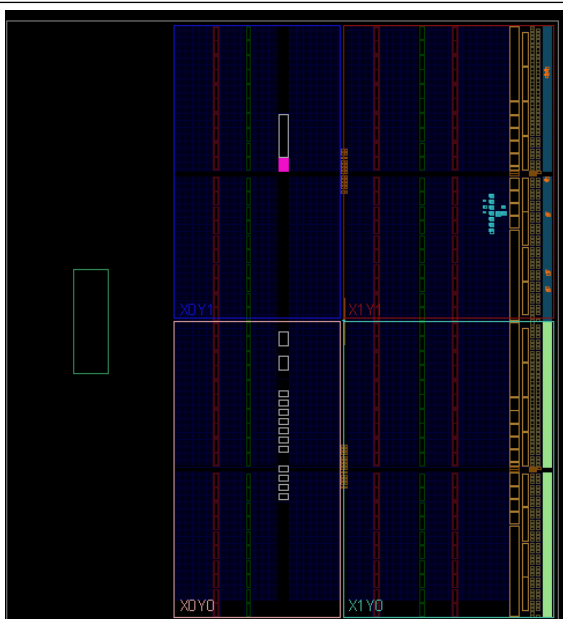
Hold

Worst Hold Slack (WHS): 0,165 ns
 Total Hold Slack (THS): 0,000 ns
 Number of Failing Endpoints: 0
 Total Number of Endpoints: 48

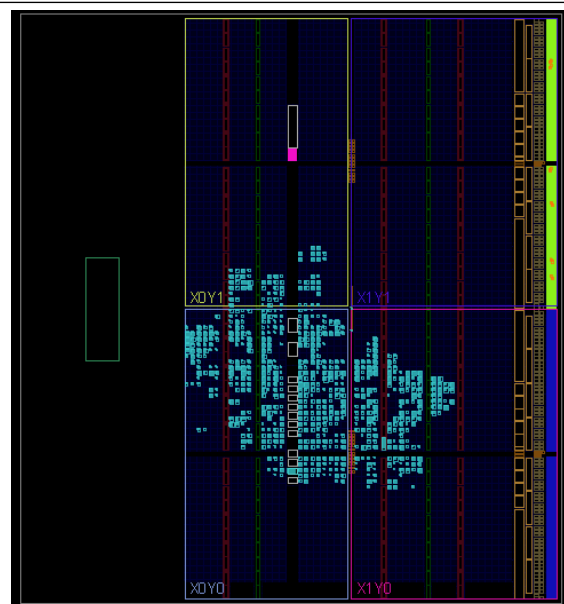
All user specified timing constraints are met.

Chemin critique (ns)	5,425
Worst negative slack (ns)	5,425
Total negative slack (ns)	0
Worst hold slack (ns)	0,165
Total hold slack (ns)	0

12. Générez le bitstream pour vérifier le système sur carte.



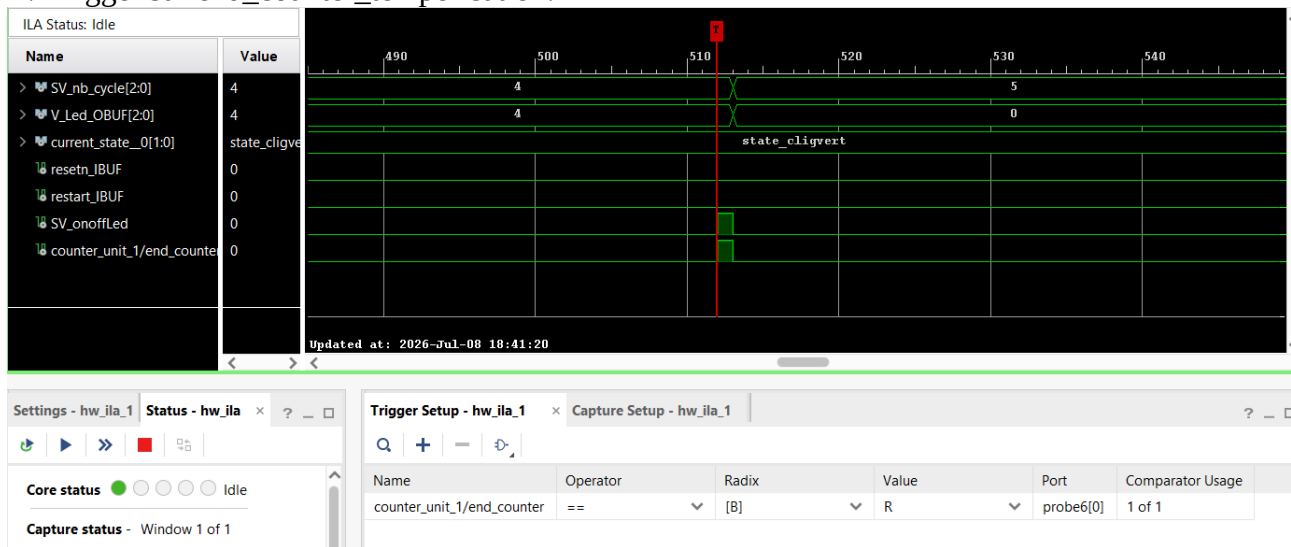
Utilisation des bascules avec un délai de temporisation de 200000000 coup d'horloge.
Sans ILA



Utilisation des bascules avec un délai de temporisation de 200000000 coup d'horloge.
Avec l'ILA

13. Vérification du bon fonctionnement à l'aide de sondes ILAs.

A/ Trigger sur end_counter_temporisation.



A chaque fin de temporisation, le système incrémente correctement le nombre de cycle.
Le Vecteur V_led passe de 4 à 0, les diodes s'éteignent, (« 100 » signifie que la led Verte est allumée) conformément à l'état dans lequel se trouve la machine d'état.

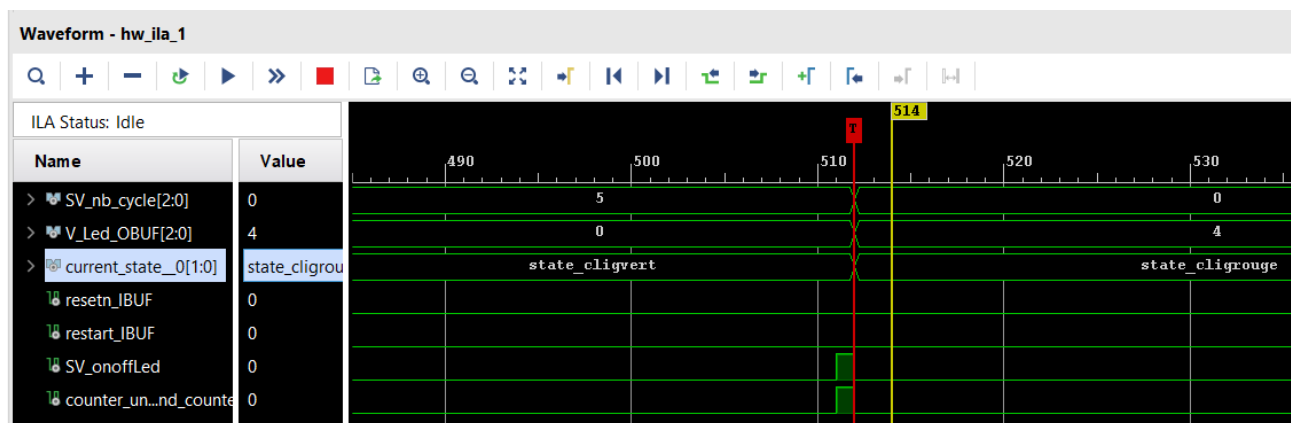
Trigger sur current_state :

S'il l'on place le trigger sur current_state 00 → Etat init, on ne trigge jamais sauf si on appuie sur le reset ou le restart

Trigger sur 01 → State_clignRouge

Trigger sur 10 → State_clignBleu

Trigger sur 11 → State_clignVert

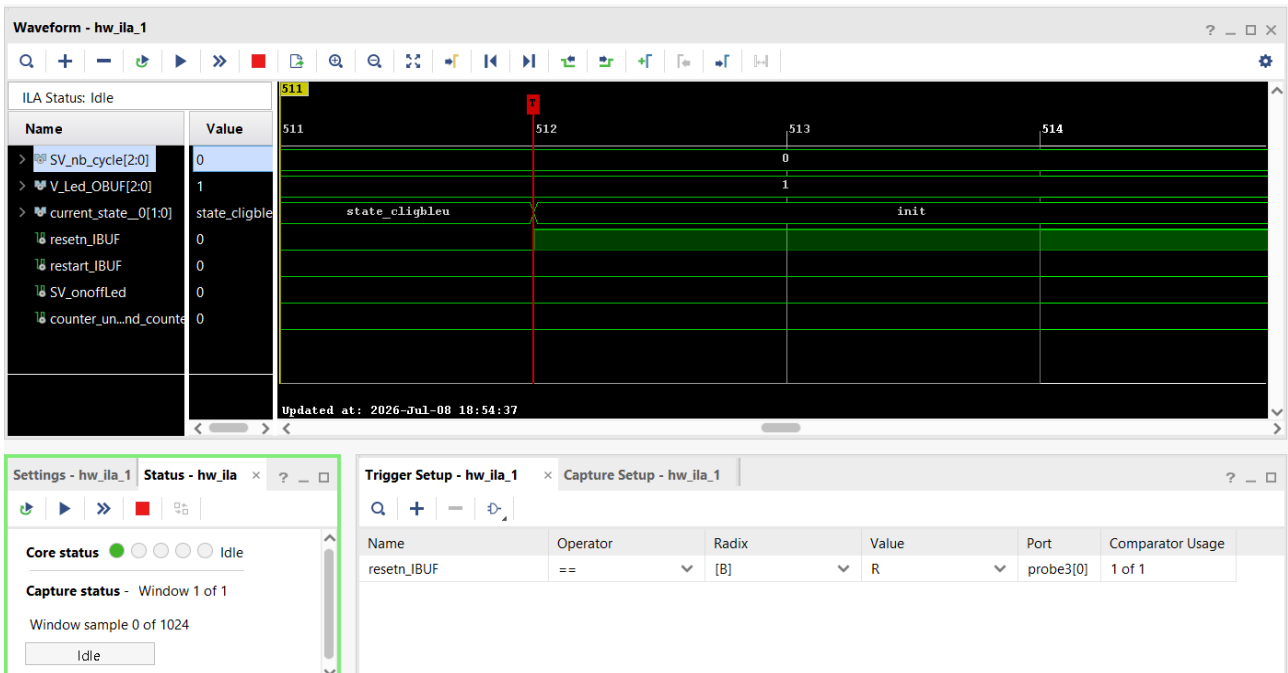


Exemple de trigger sur Current_state = 01 → Detection de l'état rouge.

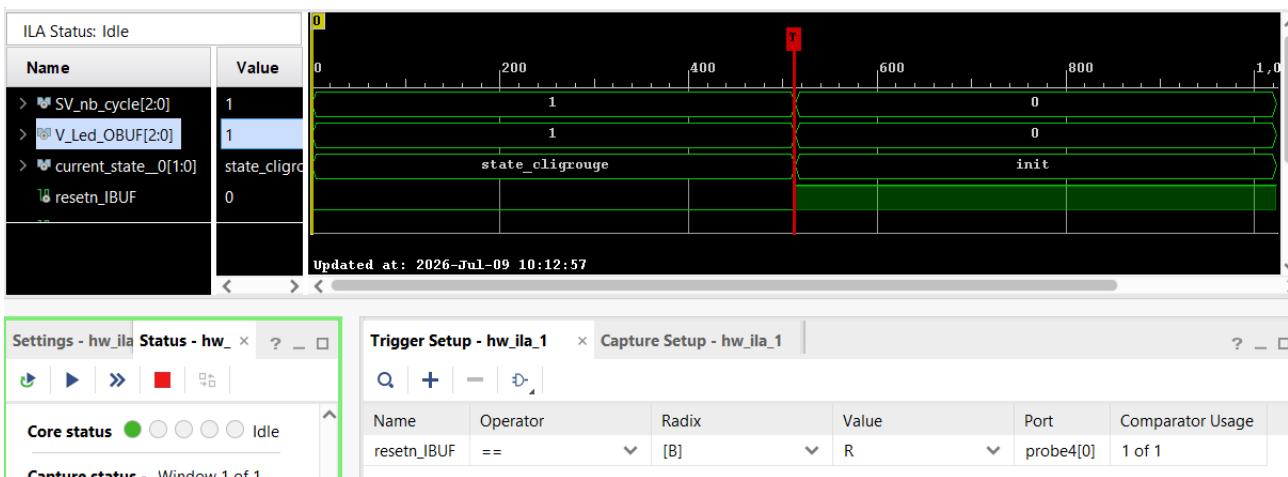
Trigger sur resetn :

Si l'on appuie sur le bouton restart, il ne se passe rien. C'est le comportement normal.

Si le bouton reset est pressé, le trigger est actif



Le current state est réinitialisé, le nb_cycle également mais le V_led_obuf conserve son état. Le système en l'état n'éteint pas les leds sur pression de bouton reset. Sur reset actif, le vecteur d'allumage des leds était mis à « 000 », ce qui prenait effet au prochain coup d'horloge, une fois le reset relâché. → Modification du programme pour éteindre directement les leds en effectant la sortie des leds à 0 sur reset haut.



Désormais, sur front montant de reset asynchrone ou restart synchrone, les diodes s'éteignent immédiatement.